

Padrões de Projeto (Command)

QXD0068 - Reuso de Software

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Introdução
- Padrão Command
- Problema
- Solução
- Analogia do Mundo Real
- Estrutura do Padrão
- Exemplo em Kotlin e Python
- Aplicabilidade
- Prós e Contras
- Implementação



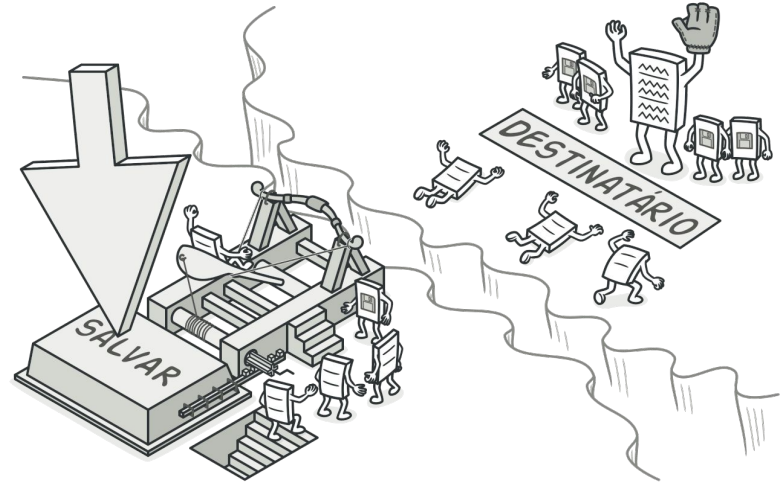
Command

Introdução

- O Command é um padrão comportamental que encapsula uma solicitação (ação, comando, tarefa) em um objeto independente, armazenando todas as informações necessárias para executá-la.
- Esse encapsulamento permite parametrizar, agendar, desfazer, refazer ou enfileirar comandos, tornando o sistema mais flexível e desacoplado.

Padrão Command

- A ideia principal é separar o remetente (quem solicita) do destinatário (quem executa).
- Cada ação do sistema é transformada em um objeto comando, que contém:
 - A referência ao destinatário (quem realiza a ação);
 - Os parâmetros necessários;
 - O método execute(), que realiza a operação.
- Assim, é possível tratar comandos de forma uniforme e genérica.



Problema

- Em sistemas com interfaces gráficas (como editores de texto), diferentes botões, menus e atalhos realizam operações similares — copiar, colar, cortar, etc.
- Uma implementação direta levaria à duplicação de código e alto acoplamento entre a interface (GUI) e a lógica de negócio.
- Toda mudança em um comportamento exigiria alterar diversas classes, violando o princípio da responsabilidade única.

Solução

- O Command propõe criar uma classe comando para cada operação.
- A interface GUI deixa de chamar diretamente os métodos da lógica de negócio — em vez disso, aciona o comando associado.
- Esse comando encapsula:
 - O objeto alvo (destinatário);
 - O método a ser executado;
 - Os parâmetros necessários.
- Dessa forma, o botão, menu ou atalho não precisa saber como a ação é executada, apenas qual comando chamar.

Analogia do Mundo Real

- Imagine um restaurante:
 - O cliente faz o pedido (remetente).
 - O garçom escreve o pedido em um papel (objeto comando).
 - O chef (destinatário) recebe o pedido e prepara o prato.

- O papel contém todas as informações necessárias (o que foi pedido, quantidade, mesa, etc.) e pode ser processado posteriormente ou em ordem.

- Assim, o garçom não precisa cozinhar — ele apenas entrega o comando.

Analogia do Mundo Real

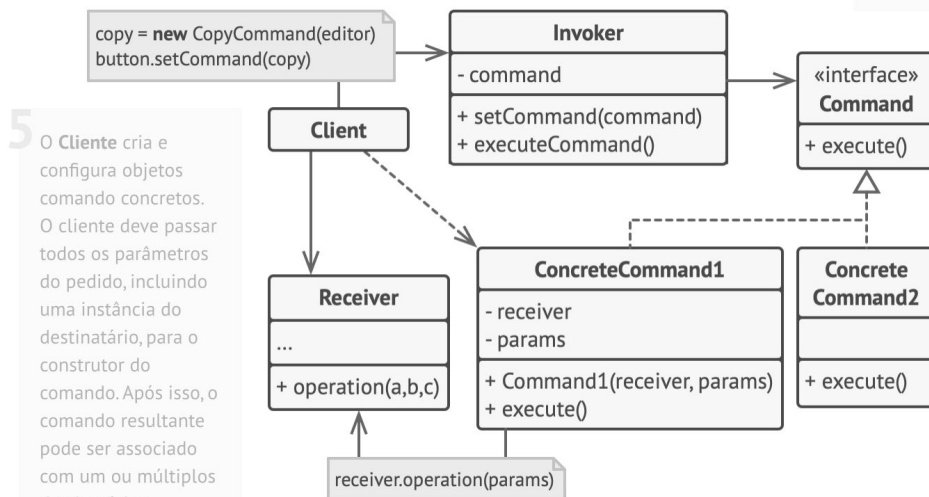


Estrutura do Padrão

- **Command (Interface)** – Define o método execute() comum a todos os comandos.
- **ConcreteCommand** – Implementa o comando específico (ex: CopyCommand, PasteCommand), armazenando o estado e o destinatário.
- **Receiver (Destinatário)** – Contém a lógica de negócio real, ou seja, quem executa o trabalho.
- **Invoker (Remetente)** – Dispara o comando (execute()), sem conhecer detalhes da implementação.
- **Client** – Cria e associa comandos e remetentes.
- Essa estrutura promove desacoplamento total entre quem solicita e quem executa.

1 A classe **Remetente** (também conhecida como *invocadora*) é responsável por iniciar os pedidos. Essa classe deve ter um campo para armazenar a referência para um objeto comando. O remetente aciona aquele comando ao invés de enviar o pedido diretamente para o destinatário. Observe que o remetente não é responsável por criar o objeto comando. Geralmente ele é pré criado através de um construtor do cliente.

2 A interface **Comando** geralmente declara apenas um único método para executar o comando.



5 O **Cliente** cria e configura objetos comando concretos. O cliente deve passar todos os parâmetros do pedido, incluindo uma instância do destinatário, para o construtor do comando. Após isso, o comando resultante pode ser associado com um ou múltiplos destinatários.

4 A classe **Destinatária** contém a lógica do negócio. Quase qualquer objeto pode servir como um destinatário. A maioria dos comandos apenas lida com os detalhes de como um pedido é passado para o destinatário, enquanto que o destinatário em si executa o verdadeiro trabalho.

3 **Comandos Concretos** implementam vários tipos de pedidos. Um comando concreto não deve realizar o trabalho por conta própria, mas passar a chamada para um dos objetos da lógica do negócio. Contudo, para simplificar o código, essas classes podem ser fundidas.

Os parâmetros necessários para executar um método em um objeto destinatário podem ser declarados como campos no comando concreto. Você pode tornar os objetos comando imutáveis ao permitir que apenas inicializem esses campos através do construtor.

```
// Interface Command
interface Command {
    fun execute()
}

// Receiver - Lógica de negócio
class Editor {
    fun copy() = println("Copiando texto...")
    fun paste() = println("Colando texto...")
}

// Concrete Commands
class CopyCommand(private val editor: Editor) : Command {
    override fun execute() = editor.copy()
}

class PasteCommand(private val editor: Editor) : Command {
    override fun execute() = editor.paste()
}

// Invoker - Botão genérico
class Button(private var command: Command) {
    fun click() = command.execute()
    fun setCommand(c: Command) { command = c }
}

// Cliente
fun main() {
    val editor = Editor()
    val copy = CopyCommand(editor)
    val paste = PasteCommand(editor)

    val button = Button(copy)
    button.click() // Copiar
    button.setCommand(paste)
    button.click() // Colar
}
```

Exemplo em Kotlin

- Editor é o **Receiver**.
- CopyCommand e PasteCommand são **ConcreteCommands**.
- Button é o **Invoker**, que dispara o comando.
- O main() é o **Client**.

```
from abc import ABC, abstractmethod

# Interface Command
class Command(ABC):
    @abstractmethod
    def execute(self):
        pass

# Receiver
class Editor:
    def copy(self):
        print("Copiando texto...")

    def paste(self):
        print("Colando texto...")

# Concrete Commands
class CopyCommand(Command):
    def __init__(self, editor):
        self.editor = editor

    def execute(self):
        self.editor.copy()

class PasteCommand(Command):
    def __init__(self, editor):
        self.editor = editor

    def execute(self):
        self.editor.paste()

# Invoker
class Button:
    def __init__(self, command):
        self.command = command

    def click(self):
        self.command.execute()
```

Exemplo em Python

Aplicabilidade

- **Use o Command quando:**
 - Deseja parametrizar objetos com operações específicas.
 - Precisa enfileirar, agendar ou executar remotamente ações.
 - Quer implementar operações reversíveis (undo/redo).
 - Deseja desacoplar remetentes e destinatários, facilitando mudanças e extensões futuras.

Prós e Contras

- **Vantagens**

- Desacopla quem solicita da execução real.
- Facilita undo/redo e histórico de ações.
- Permite filas e agendamento de comandos.
- Segue os princípios SRP e Aberto/Fechado.
- Permite combinar comandos complexos a partir de simples.

- **Desvantagens**

- Adiciona camada extra de abstração, aumentando o número de classes.
- Pode gerar complexidade desnecessária em sistemas simples.



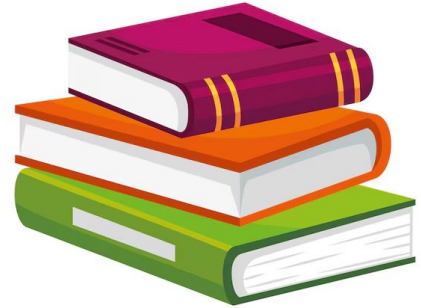
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

